QISKIT FALL FEST 2026 — ADVANCED TRACK

Quantum Boss Battle

Technical Report: Gamifying Low-Overhead Quantum Error Detection

Team: Qubit Titans
Authors: Nikkhil Saini, Lokesh Bhoi Mali, Ranjeet Nishad
Repository: https://github.com/nnikhilsaini55-byte/quantum-boss-battle

PRELIMINARY VERSION — benchmark notebook not run/verified

Submission note: This version intentionally does not present unverified numerical results as confirmed. Complete the benchmark table after a successful reproducible run.

Abstract

Quantum Boss Battle is a browser-based educational game intended to make low-overhead quantum error detection easier to understand through a battle-style interface. The project is aligned with the Qiskit Fall Fest 2026 Advanced Track, which emphasizes a real Qiskit-based error-detection workflow and evaluation using fidelity, retained-shot fraction, and circuit overhead.

This version of the report is deliberately preliminary: the game source is available in the public repository and the game has been reported as opening and playable, but the experiment notebook advanced_track_experiments.ipynb has not been successfully executed during this verification pass. Therefore, no numerical benchmark is presented as a confirmed result. Saved notebook outputs, if any, must be treated as unverified until rerun and reconciled with the source code and README.

1. Problem Statement and Motivation

Quantum circuits are sensitive to gate and measurement noise. Full quantum error correction can require encoding, repeated syndrome extraction, and decoding. Low-overhead error detection offers a narrower learning task: introduce checks that flag some corrupted executions, then discard shots with non-trivial syndromes. Detection does not repair the underlying state and cannot identify every possible error.

Quantum Boss Battle aims to turn this trade-off into an interactive experience while preserving the distinction between error detection, error mitigation, and error correction.

2. Objectives

The project aims to help players understand: (1) error detection versus mitigation and correction; (2) checks, ancilla measurements, syndromes, and post-selection; (3) the effects of noise, circuit depth, check placement, and non-Clifford gates; and (4) the trade-off between accepted-output quality, retained shots, and resource overhead.

A core requirement is that scientific metrics and any score described as quantum-derived must come from executed Qiskit results rather than manually assigned values.

3. Technical Background

Error detection uses checks or auxiliary measurements to flag executions that do not satisfy expected conditions. Post-selection retains accepted shots and discards shots with non-trivial syndromes. Error mitigation changes estimates

during post-processing, while error correction uses encoded information and decoding to infer and apply corrections. These are different techniques and should not be described interchangeably.

The project brief focuses on error detection and asks teams to compare circuit configurations while reporting both fidelity and the fraction of shots retained.

4. System Overview and Game Mechanics

The repository describes a browser interface with a backend API and a Qiskit simulation layer. The game interface uses Attack and Defend actions. This report does not assume that either action performs a particular quantum operation unless that behavior is confirmed in the current source and demonstrated in a successful run.

The intended flow is: player action → backend request → circuit construction and noise configuration → local Qiskit Aer execution → metric calculation → game feedback. The implementation should be checked to confirm that the backend is called successfully and that displayed scores are derived from simulation outputs.

5. Simulation Methodology

The planned evaluation uses local Qiskit Aer simulation. A reproducible experiment should specify the payload circuit, circuit width and depth, shot count, noise model and parameters, check count and placement, ancilla count, additional two-qubit gates, depth increase, random seed, and package versions.

For Clifford-only circuits, qiskit-paulice may be used through its supported API to add Pauli checks. Its automated check insertion is limited to Clifford circuits; it must not be applied to circuits containing T gates or Rz(π/4). T-doped circuits should be evaluated without those automated checks, or with explicitly designed checks only where the relevant Pauli action commutes with the T gate (I or Z on that qubit at that point). The actual source code must be inspected to confirm which method is implemented.

The ideal reference state and fidelity definition must be documented exactly as implemented. If no shots survive post-selection, the report should state that the retained-shot fraction is zero and should not present a misleading post-selected fidelity.

6. Evaluation Metrics

Raw fidelity: fidelity between the ideal reference and noisy output before post-selection, using the exact implementation's definition.

Post-selected fidelity: fidelity computed after filtering shots using the accepted syndrome condition.

Retained-shot fraction: accepted shots divided by total executed shots. Report the total and accepted counts as well as the fraction.

Ancilla overhead: number of additional ancilla qubits beyond the payload circuit.

Added circuit cost: additional two-qubit gates and the change in circuit depth.

Post-selected fidelity must always be reported alongside retained-shot fraction. If no checks are used, explain how raw and post-selected metrics relate in the implementation.

7. Experimental Results and Verification Status

Verification status: NOT VERIFIED. The notebook advanced_track_experiments.ipynb has not been successfully run for this report. Consequently, this version does not claim a measured winner, does not certify saved notebook output, and does not present numerical benchmark values as confirmed.

Prior saved outputs may be useful as leads, but the README and notebook outputs have been reported to disagree, and some retained-shot/discarded-shot figures do not reconcile with the stated total shot count. Those values must not be copied into the final results as though verified.

Complete the table below after the notebook or benchmark script runs successfully. For every row, preserve the actual output and include total shots, accepted shots, noise settings, circuit width/depth, check configuration, and run identifier.

8. Benchmark Table to Complete

Use the table as a recording template only. Replace each PENDING field with values printed by a successful run; do not estimate or manually invent values.

9. Discussion and Limitations

Because benchmark execution is pending, this report does not yet identify the best configuration or draw quantitative conclusions about fidelity improvement. Once results are verified, discuss whether additional checks improve post-selected fidelity enough to justify discarded shots and added circuit cost.

Local Aer simulations depend on the selected synthetic noise model, finite shot count, circuit design, and check placement. They do not fully reproduce all effects of physical quantum hardware. Automated qiskit-paulice check insertion is limited to Clifford circuits; T-doped circuits need separate handling. The report must match the actual implementation and should not imply that hardware execution occurred unless it did.

10. Reproducibility and Repository

Public repository: https://github.com/nnikhilsaini55-byte/quantum-boss-battle

The repository should include runnable source code, this written report, and the presentation video. The exact environment setup, frontend/backend launch commands, Python/Qiskit versions, benchmark command, and random seeds should be copied from the repository and tested from a clean environment before submission.

Current limitation: the notebook execution and benchmark outputs have not been verified for this report. This preliminary report is transparent about that status but does not substitute for measured results if the judging criteria require completed experiments.

11. Conclusion

Quantum Boss Battle aims to teach that error detection is a trade-off rather than a repair mechanism: checks may improve the quality of accepted outputs, but they consume resources and discard executions. A defensible final evaluation must show reproducible Qiskit results for raw fidelity, post-selected fidelity, retained-shot fraction, and circuit overhead. The project's quantitative conclusion remains pending successful benchmark execution.

References

1. W. Sun, "QFF 2026 Hackathon: Gamifying Error Detection, Mitigation and Correction," prompt provided for Qiskit Fall Fest 2026.

2. IBM Quantum / Qiskit Community, qiskit-paulice documentation and low-overhead error detection with spacetime codes tutorial. Add the exact resource URLs and access date used in the final submission.

3. D. Litinski, "A Game of Surface Codes," Quantum, vol. 3, p. 128, 2019.

4. Error Correction Zoo, "Coherent-parity-check (CPC) code," https://errorcorrectionzoo.org/c/cpc.

5. IBM Quantum Learning, “Foundations of quantum error correction.” Add the exact resource URL and access date used in the final submission.

Benchmark recording template

Configuration	Checks	Raw fidelity	Post-selected fidelity	Total / accepted shots	Retained fraction	Ancillas	Added 2Q gates / depth
Clifford baseline	0	PENDING	PENDING / N.A.	PENDING	PENDING	PENDING	PENDING
Clifford + checks	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
Clifford alternate checks	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
T-doped variant	No automated Paulice checks / valid hand check only	PENDING	PENDING / N.A.	PENDING	PENDING	PENDING	PENDING

Consistency check: accepted shots + discarded shots must equal total executed shots. Retained fraction = accepted shots ÷ total executed shots. Do not enter estimated values.

AI-use disclosure

AI assistance was used to help organize and edit the report text. Project implementation claims and numerical results must be checked against the team’s source code and successful benchmark output. The team remains responsible for the correctness of the submitted code, measurements, and report.